

Aleatoric and epistemic uncertainty representations and their evaluation

Journal club 26/11/2025

Mira Jürgens

PhD student

Department of Data Science and Mathematical Modeling

Ghent University

Promotor: Willem Waegeman



Common machine learning models are often overconfident.



AI Model prediction



99% No tumor

Common machine learning models are often overconfident.



AI Model prediction



99% No tumor

Expert consensus



60% No tumor

40% Tumor

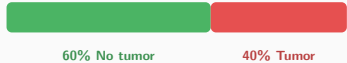
Common machine learning models are often overconfident.



AI Model prediction



Expert consensus



Aleatoric and epistemic uncertainty in machine learning

Aleatoric vs epistemic uncertainty

Assume we have a probabilistic model $f : \mathcal{X} \rightarrow \mathbb{P}(\mathcal{Y})$ that estimates the conditional distribution $\mathbb{P}_{Y|X}$ of a r.v. Y given X . We can distinguish between **two types of uncertainty**:

Aleatoric vs epistemic uncertainty

Assume we have a probabilistic model $f : \mathcal{X} \rightarrow \mathbb{P}(\mathcal{Y})$ that estimates the conditional distribution $\mathbb{P}_{Y|X}$ of a r.v. Y given X . We can distinguish between **two types of uncertainty**:

Aleatoric uncertainty

irreducible randomness in $\mathbb{P}_{Y|X}$



Aleatoric vs epistemic uncertainty

Assume we have a probabilistic model $f : \mathcal{X} \rightarrow \mathbb{P}(\mathcal{Y})$ that estimates the conditional distribution $\mathbb{P}_{Y|X}$ of a r.v. Y given X . We can distinguish between **two types of uncertainty**:

Aleatoric uncertainty

irreducible randomness in $\mathbb{P}_{Y|X}$



Epistemic uncertainty

reducible ignorance about the true $\mathbb{P}_{Y|X}$

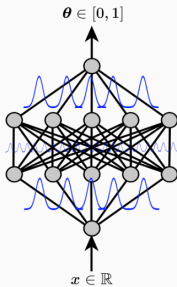


Different representations of epistemic uncertainty

Distribution-based

second-order distribution $q(\theta|x)$

- e.g. Bayesian neural networks, evidential models, ensembles (approx.)

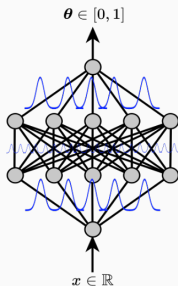


Different representations of epistemic uncertainty

Distribution-based

second-order distribution $q(\theta|x)$

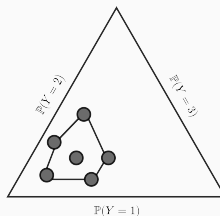
- e.g. Bayesian neural networks, evidential models, ensembles (approx.)



Set-based

(convex) set $\mathcal{S}$ of probability distributions

- e.g. ensemble models, credal/interval networks

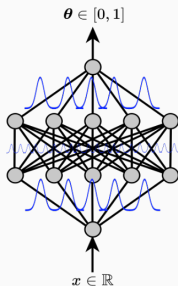


Different representations of epistemic uncertainty

Distribution-based

second-order distribution $q(\theta|x)$

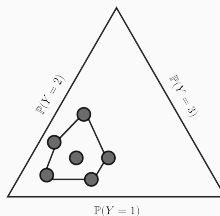
- e.g. Bayesian neural networks, evidential models, ensembles (approx.)



Set-based

(convex) set $\mathcal{S}$ of probability distributions

- e.g. ensemble models, credal/interval networks



When do we know that a given representation is a *valid* representation of EU?

Notation

Symbol	Meaning
<i>General Supervised Learning</i>	
$\mathcal{X} = \mathbb{R}^D$	Instance space
$\mathcal{Y}$	Target space (e.g., $\{1, \dots, K\}$ or $\mathbb{R}$)
$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$	Dataset of N i.i.d. realizations
<i>Probabilistic Modeling</i>	
$p(y \mathbf{x}, \theta)$	Conditional likelihood (e.g., Categorical or Gaussian)
$\theta \in \Theta$	Distributional parameters (e.g., $\Theta = \Delta_K$ or $\mathbb{R} \times \mathbb{R}_+$)
$f_\phi : \mathcal{X} \rightarrow \Theta$	Neural network parameterized by weights ϕ
<i>Uncertainty Representations</i>	
$\mathcal{F} = \{f_{\phi_m}\}_{m=1}^M$	Ensemble of M predictors (Set-based)
$q(\theta \mathbf{x})$	Second-order distribution over θ (e.g., Dirichlet)

1. Set-based epistemic uncertainty representations

Machine Learning (2025) 114:202
<https://doi.org/10.1007/s10994-025-06844-8>

A calibration test for evaluating set-based epistemic uncertainty representations

Mira Jürgens¹ · Thomas Mortier² · Eyke Hüllermeier³ · Viktor Bengs³ · Willem Waegeman¹

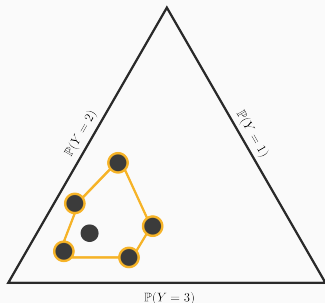
Received: 9 June 2025 / Revised: 14 July 2025 / Accepted: 15 July 2025 /
Published online: 5 August 2025
© The Author(s) 2025

Main research questions:

- *validity* for sets of probability distributions
- role of *instance-dependency*
- calibration as a *necessary criterion* for validity

When is a set-based representation valid?

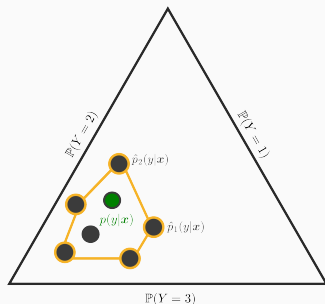
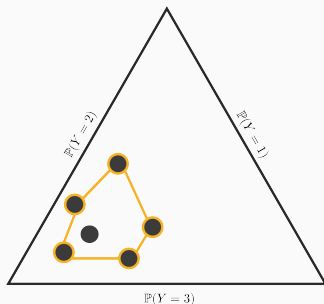
For each instance $x \in \mathcal{X}$ define $\mathcal{S}(x)$ as the **convex set of predicted probability distributions**.



When is a set-based representation valid?

For each instance $x \in \mathcal{X}$ define $\mathcal{S}(x)$ as the **convex set of predicted probability distributions**.

We say that $\mathcal{S}$ is a valid representation if $p(y|\mathbf{x}) \in \mathcal{S}(x)$ for all $\mathbf{x} \in \mathcal{X}$.



How do we define $\mathcal{S}(x)$?

Pragmatic approach: We look at the convex set of convex combinations.

How do we define $\mathcal{S}(x)$?

Pragmatic approach: We look at the convex set of convex combinations.

Credal set formed by ensemble model

Let $\mathcal{F} = \{f_{\phi_m}\}_{m=1}^M$ be a set of M different probabilistic classifiers.

Define

$$\mathcal{S}(\mathcal{F}, x) = \{f_{\lambda}(x) \in \Delta_K \mid f_{\lambda}(x) = \sum_{i=1}^M \lambda_i(x) f^{(i)}(x)\}$$

as the set of all **convex combinations**, where $\lambda : \mathcal{X} \rightarrow \Delta_M$ is a vector-valued function **from the instance space** to the M -simplex.

How do we define $\mathcal{S}(x)$?

Pragmatic approach: We look at the convex set of convex combinations.

Credal set formed by ensemble model

Let $\mathcal{F} = \{f_{\phi_m}\}_{m=1}^M$ be a set of M different probabilistic classifiers.

Define

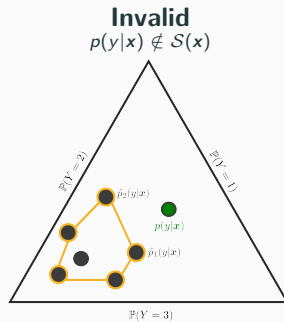
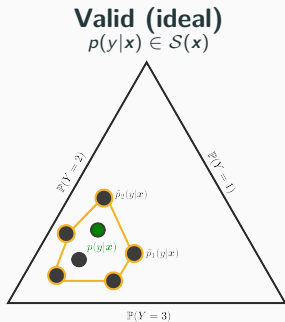
$$\mathcal{S}(\mathcal{F}, x) = \{f_{\lambda}(x) \in \Delta_K \mid f_{\lambda}(x) = \sum_{i=1}^M \lambda_i(x) f^{(i)}(x)\}$$

as the set of all **convex combinations**, where $\lambda : \mathcal{X} \rightarrow \Delta_M$ is a vector-valued function **from the instance space** to the M -simplex.

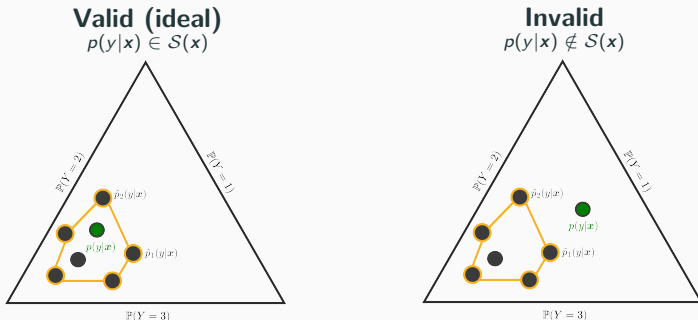
We say that $\mathcal{F}$ is *valid* iff

$$p(y|\mathbf{x}) \in \mathcal{S}(\mathcal{F}, \mathbf{x}) \quad \text{for almost all } \mathbf{x} \in \mathcal{X}.$$

How can we test that in practice?



How can we test that in practice?

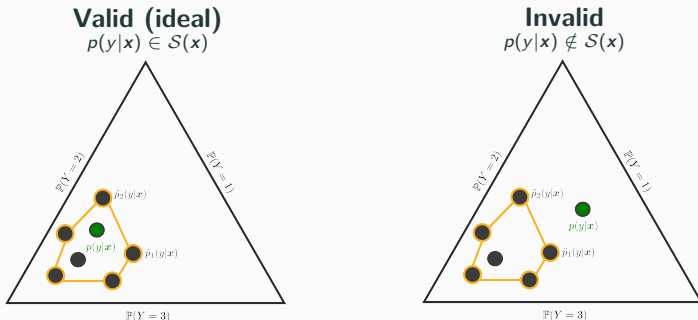


Typically not possible!

In classical settings, we **cannot observe or sample from** $\mathbb{P}_{Y|X=x}$.

We therefore test the ***necessary*** condition:

How can we test that in practice?



Typically not possible!

In classical settings, we **cannot observe or sample from** $\mathbb{P}_{Y|X=x}$.

We therefore test the ***necessary*** condition:

$$H_0 : \forall x \exists \lambda(x) \in \Delta_M \text{ s.t. } f_\lambda \text{ is calibrated} \quad \text{vs} \quad H_1 : \nexists \lambda(x).$$

Calibration as a distribution matching problem

Distribution calibration

A classifier $f : \mathcal{X} \rightarrow \Delta^K$ is calibrated (in distribution) iff

$$\mathbb{P}(Y = k \mid f(X)) = f_k(X) \quad \text{a.s. for all } k.$$

How do we measure calibration? **The Challenge:** We must aggregate data in a **local neighborhood**.

Calibration as a distribution matching problem

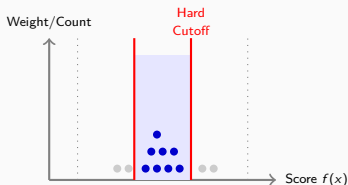
Distribution calibration

A classifier $f : \mathcal{X} \rightarrow \Delta^K$ is calibrated (in distribution) iff

$$\mathbb{P}(Y = k \mid f(X)) = f_k(X) \quad \text{a.s. for all } k.$$

How do we measure calibration? **The Challenge:** We must aggregate data in a **local neighborhood**.

1. Binning



Hard boundaries (Discontinuous)

Non-differentiable

Calibration as a distribution matching problem

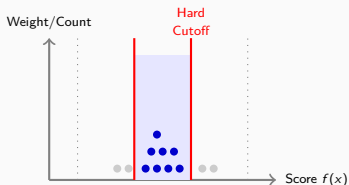
Distribution calibration

A classifier $f : \mathcal{X} \rightarrow \Delta^K$ is calibrated (in distribution) iff

$$\mathbb{P}(Y = k \mid f(X)) = f_k(X) \quad \text{a.s. for all } k.$$

How do we measure calibration? **The Challenge:** We must aggregate data in a **local neighborhood**.

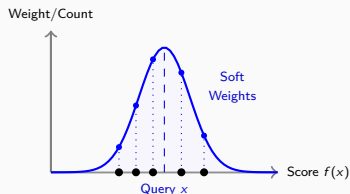
1. Binning



Hard boundaries (Discontinuous)

Non-differentiable

2. Kernels



✓ Smooth weighting

✓ Differentiable

Two families of kernel-based calibration estimators

1. Expectation-over-Divergence (EoD)

$$\text{CE}(f) = \mathbb{E} \left[d \left(\mathbb{E}[\mathbf{1}\{Y = k\} \mid f(X)], f(X) \right) \right]$$

- $d = \|\cdot\|_{L_2} \Rightarrow \text{CE}_2$ ¹
- $d = d_{\text{KL}} \Rightarrow \text{CE}_{\text{KL}}$ ²

¹Popordanoska et al., “A consistent and differentiable lp canonical calibration error estimator”.

²Popordanoska et al., “Consistent and asymptotically unbiased estimation of proper calibration errors”.

³Widmann et al., “Calibration tests in multi-class classification: A unifying framework”.

⁴Kumar et al., “Trainable calibration measures for neural networks from kernel mean embeddings”.

Two families of kernel-based calibration estimators

1. Expectation-over-Divergence (EoD)

$$\text{CE}(f) = \mathbb{E} \left[d \left(\mathbb{E}[\mathbf{1}\{Y = k\} \mid f(X)], f(X) \right) \right]$$

- $d = \|\cdot\|_{L_2} \Rightarrow \text{CE}_2^1$
- $d = d_{\text{KL}} \Rightarrow \text{CE}_{\text{KL}}^2$

2. Integral Probability Metrics (IPM)

$$\text{CE}(f) = \sup_{\phi \in \mathcal{H}} \left| \mathbb{E}[\phi(f(X), Y)] - \mathbb{E}[\phi(f(X), Z)] \right|$$

with $\mathcal{H}$ being an RKHS admitting a kernel $k \Rightarrow \text{CE}_k^3, \text{CE}_{\text{MMD}}^4$

¹Popordanoska et al., “A consistent and differentiable lp canonical calibration error estimator”.

²Popordanoska et al., “Consistent and asymptotically unbiased estimation of proper calibration errors”.

³Widmann et al., “Calibration tests in multi-class classification: A unifying framework”.

⁴Kumar et al., “Trainable calibration measures for neural networks from kernel mean embeddings”.

Calibration estimators are random variables, not just numbers

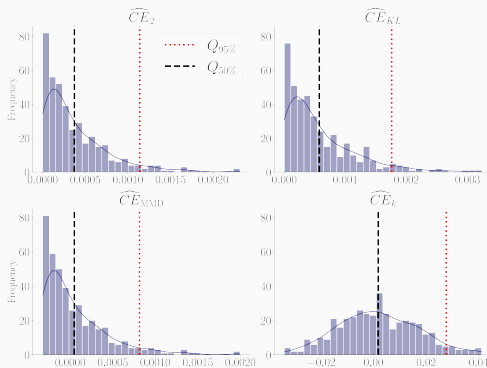
Under H_0 (perfect calibration)
their distribution has:

- different shapes,
- different scales.

Implication for testing calibration

We can estimate the critical value $Q_{95\%}$ (the red line) via resampling.^a

^aVaicenavicius et al., "Evaluating model calibration in classification".



Testing calibration as an optimization problem

Goal: Find a calibrated convex combination

We want to know if *any* convex combination exists that yields a calibrated model. This naturally leads to the question of

$$\min_{\lambda: \mathcal{X} \rightarrow \Delta_M} \underbrace{g(f_\lambda)}_{\text{calibration error}} \stackrel{?}{=} 0$$

Testing calibration as an optimization problem

Goal: Find a calibrated convex combination

We want to know if *any* convex combination exists that yields a calibrated model. This naturally leads to the question of

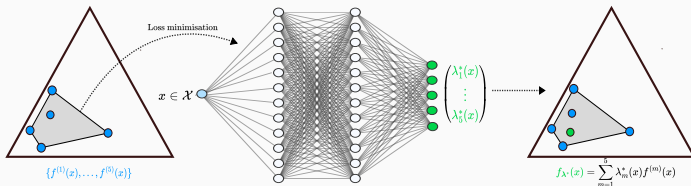
$$\min_{\lambda: \mathcal{X} \rightarrow \Delta_M} \underbrace{g(f_\lambda)}_{\text{calibration error}} \stackrel{?}{=} 0$$

Why direct optimization fails

Minimizing g directly is **ill-posed**:

1. **Non-Convexity:** not all calibration estimators are convex.
2. **Trivial Solutions:** there are typically many calibrated convex combinations.

Our approach: A neural calibration network



Composite Loss Function

To avoid trivial solutions, we optimize a joint objective:

$$\lambda^* = \arg \min_{\lambda} \left(\underbrace{\hat{\mathbb{E}}[\ell(Y, f_{\lambda}(X))]}_{\text{expected risk}} + \gamma \cdot \underbrace{g(f_{\lambda})}_{\text{calibration error}} \right).$$

- proper scoring rule ℓ : avoids trivial solutions
- calibration error g : explicitly minimizes the chosen metric (e.g., CE_{MMD}).

Algorithm 1 - Testing calibration via optimisation

- 1: **Input:** calibration dataset $\mathcal{D}_{val}$, set of pred. probability distributions $\mathcal{F}$, significance level α , calibration error estimator $\hat{g}$

Algorithm 1 - Testing calibration via optimisation

- 1: **Input:** calibration dataset $\mathcal{D}_{val}$, set of pred. probability distributions $\mathcal{F}$, significance level α , calibration error estimator $\hat{g}$

1. Optimization — learn "best possible" $\lambda(x)$

1: $\Lambda^* \leftarrow \arg \min_{\Lambda} \mathcal{L}(f_{\Lambda}, \mathcal{D}_{val})$

2: $f_{\Lambda^*} \leftarrow \Lambda^* \cdot \mathcal{F}$

▷ convex combination

Algorithm 1 - Testing calibration via optimisation

- 1: **Input:** calibration dataset $\mathcal{D}_{val}$, set of pred. probability distributions $\mathcal{F}$, significance level α , calibration error estimator $\hat{g}$

1. Optimization — learn "best possible" $\lambda(x)$

1: $\Lambda^* \leftarrow \arg \min_{\Lambda} \mathcal{L}(f_{\Lambda}, \mathcal{D}_{val})$

2: $f_{\Lambda^*} \leftarrow \Lambda^* \cdot \mathcal{F}$

▷ convex combination

2. Bootstrap — simulate labels under H_0

1: $t \leftarrow \hat{g}(f_{\Lambda^*}, \mathcal{D}_{val})$

▷ observed calibration error

2: **for** $d = 1, \dots, D$ **do**

3: $\tilde{Y} \sim f_{\Lambda^*}(X)$

▷ **Key:** sample labels from *calibrated* model

4: $\tilde{\mathcal{D}} \leftarrow (X, \tilde{Y})$

▷ create synthetic dataset with new labels

5: $t_{0,d} \leftarrow \hat{g}(f_{\Lambda^*}, \tilde{\mathcal{D}})$

Algorithm 1 - Testing calibration via optimisation

- 1: **Input:** calibration dataset $\mathcal{D}_{val}$, set of pred. probability distributions $\mathcal{F}$, significance level α , calibration error estimator $\hat{g}$

1. Optimization — learn "best possible" $\lambda(x)$

- 1: $\Lambda^* \leftarrow \arg \min_{\Lambda} \mathcal{L}(f_{\Lambda}, \mathcal{D}_{val})$
2: $f_{\Lambda^*} \leftarrow \Lambda^* \cdot \mathcal{F}$ ▷ convex combination

2. Bootstrap — simulate labels under H_0

- 1: $t \leftarrow \hat{g}(f_{\Lambda^*}, \mathcal{D}_{val})$ ▷ observed calibration error
2: **for** $d = 1, \dots, D$ **do**
3: $\tilde{Y} \sim f_{\Lambda^*}(X)$ ▷ **Key:** sample labels from *calibrated* model
4: $\tilde{D} \leftarrow (X, \tilde{Y})$ ▷ create synthetic dataset with new labels
5: $t_{0,d} \leftarrow \hat{g}(f_{\Lambda^*}, \tilde{D})$

- 1: $q_{1-\alpha} \leftarrow (1 - \alpha)$ -quantile of $\{t_{0,d}\}_{d=1}^D$
2: **If** $t > q_{1-\alpha}$ **then reject** H_0 **else do not reject**

Experimental setup: synthetic data

Hypotheses

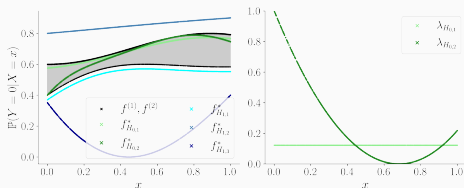
H_0 Valid combination exists

- $H_{0,1}$: constant weights
- $H_{0,2}$: instance-dependent weights

H_1 No valid combination

Varying distance γ to the boundary:

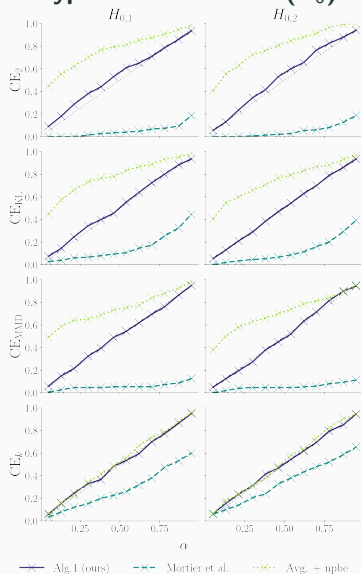
- $H_{1,1}$: $\gamma = 0.05$
- $H_{1,2}$: $\gamma = 0.2$
- $H_{1,3}$: $\gamma = 0.8$



Binary classification setup - probability simplex (left) and $\lambda(x)$ for the case where H_0 is true.

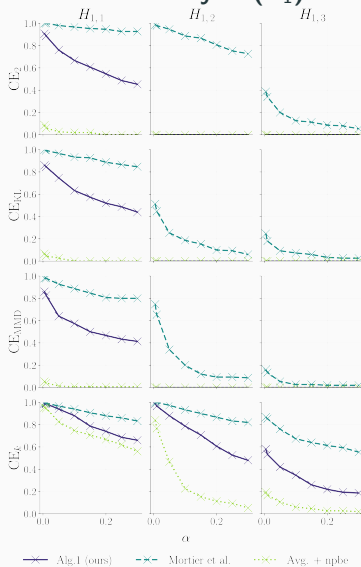
Synthetic Results: Type-I Error and Power

Type-I error control (H_0)



x-axis: α y-axis: rejection rate

Power analysis (H_1)



x-axis: α y-axis: (1 - rejection rate)

Showcase on real data - CIFAR-10/100

Tests on Deep Ensemble, MC Dropout for VGG-16, ResNet-18 architectures.

In general:

- results on ensemble models (DE, MC Dropout) show very close proximity to the calibration test applied to mean estimate
- deep ensembles are more often calibrated than MC Dropout
- less complex model (ResNet-18) tends to be more often calibrated than more complex model (VGG-16).

Summary: Validating set-based epistemic uncertainty representations remains a difficult task, but calibration can provide a useful criterion.

Key Findings

- **Method:** optimisation based approach yields test with higher power than previous approaches, asymptotic Type 1 error control.
- **Usage:** flag invalidity of set based representations.

Limitations

- Calibration is a *necessary* but not *sufficient* criterion.
- possible overparametrization due to instance-dependency (trade-off data efficiency - expressiveness)



Code



Paper

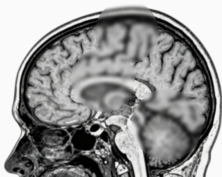
Outlook

- Extension to regression setting
- local calibration neighborhoods

2. Disentanglement of Aleatoric and Epistemic Uncertainty

Why do we need uncertainty disentanglement?

Recall the MRI example: Why could the model be uncertain?



Senario A: aleatoric

- The image is inherently noisy/ambiguous.
- **Action:** Improve sensor hardware. More data does not reduce uncertainty.



Scenario B: epistemic

- The anatomy is of rare type/unseen.
- **Action:** Collect more training data. The model can be improved.

Part 2: Evaluating distribution-based epistemic uncertainty representations

Why machine learning models fail to fully capture
epistemic uncertainty

Sebastián Jimenez^{1*}, Mira Jürgens^{1†} and Willem Waegeman¹

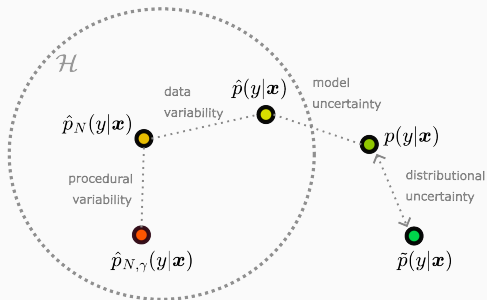
^{1*}Department of Data Analysis and Mathematical Modeling, Ghent
University, Coupure Links 653, Ghent, 9000, Belgium.

We raise **three main concerns**:

1. It is questionable which *source* of epistemic uncertainty common uncertainty quantification methods capture.
2. We need more proper *evaluation protocols* for epistemic uncertainty.
3. Proper AU estimation is only possible if EU is sufficiently low.

Sources of epistemic uncertainty

Define $\mathcal{D}_N \sim P^N$ $\gamma \sim P^\gamma$ as the random variables reflecting data- and procedural variability.



Sources of epistemic uncertainty. Figure adapted from Huang et al.^a

^aHuang et al., “Efficient uncertainty quantification and reduction for over-parameterized neural networks”.

1. model uncertainty:

due to limited hypothesis space $\mathcal{H}$ (systematic bias).

2. estimation uncertainty:

- **data uncertainty:** dataset $\mathcal{D}_N \sim P^N$.
- **procedural uncertainty:** seeds $\gamma \sim P^\gamma$.

3. distributional uncertainty:

due to distribution shifts.

Uncertainty from a generalized bias-variance perspective

Let ℓ be a **proper scoring rule** (e.g., brier score, log loss). The expected risk of a predictive distribution $\hat{p}(y|\mathbf{x})$ decomposes as⁵

Decomposition of pointwise expected loss

$$\mathbb{E}_{y|\mathbf{x}}[\ell(\hat{p}(y|\mathbf{x}), y)] = \underbrace{\mathbb{H}_\ell[p(y|\mathbf{x})]}_{\substack{\text{aleatoric} \\ \text{(generalized entropy)}}} + \underbrace{D_\ell(p(y|\mathbf{x}), \hat{p}(y|\mathbf{x}))}_{\substack{\text{epistemic} \\ \text{(divergence between } \hat{p} \text{ and } p)}}$$

⁵Gruber et al., “Sources of Uncertainty in Machine Learning—A Statisticians’ View”.

⁶Pfau, “A generalized bias-variance decomposition for bregman divergences”.

Uncertainty from a generalized bias-variance perspective

Let ℓ be a **proper scoring rule** (e.g., brier score, log loss). The expected risk of a predictive distribution $\hat{p}(y|\mathbf{x})$ decomposes as⁵

Decomposition of pointwise expected loss

$$\mathbb{E}_{y|\mathbf{x}}[\ell(\hat{p}(y|\mathbf{x}), y)] = \underbrace{\mathbb{H}_\ell[p(y|\mathbf{x})]}_{\substack{\text{aleatoric} \\ \text{(generalized entropy)}}} + \underbrace{D_\ell(p(y|\mathbf{x}), \hat{p}(y|\mathbf{x}))}_{\substack{\text{epistemic} \\ \text{(divergence between } \hat{p} \text{ and } p)}}$$

The expectation of D_ℓ over both data- and procedural uncertainty further decomposes into a **generalized bias** and a **variance** term⁶:

$$\mathbb{E}_{\mathcal{D}_N, \gamma}[D_\ell(p, \hat{p}_{N, \gamma})] = \underbrace{D_\ell(p, \bar{p}^*)}_{\substack{\text{bias} \\ \text{(systematic + search bias)}}} + \underbrace{\mathbb{E}_{\mathcal{D}_N, \gamma}[D_\ell(\bar{p}^*, \hat{p}_{N, \gamma})]}_{\text{variance}}.$$

⁵Gruber et al., “Sources of Uncertainty in Machine Learning—A Statisticians’ View”.

⁶Pfau, “A generalized bias-variance decomposition for bregman divergences”.

Concrete illustration for the regression case

Assume $y_i = f(\mathbf{x}) + \epsilon_i$, where $\epsilon_i \sim \mathcal{N}(0, \sigma^2(\mathbf{x}_i))$. For $\ell_{MSE}(\hat{p}, y) = (\hat{y} - y)^2$, the decomposition attains a familiar form:

Aleatoric-epistemic uncertainty decomposition for squared error regression

$$\mathbb{E}_{y|\mathbf{x}}[\mathbb{E}_{\mathcal{D}_N, \gamma}(\hat{y} - y)^2|\mathbf{x}] = \underbrace{\sigma^2(\mathbf{x})}_{\text{aleatoric}} + \underbrace{\text{Var}_{\mathcal{D}_N, \gamma}(\hat{y}|\mathbf{x}) + \text{bias}^2(\hat{y}|\mathbf{x})}_{\text{epistemic}}$$

Concrete illustration for the regression case

Assume $y_i = f(\mathbf{x}) + \epsilon_i$, where $\epsilon_i \sim \mathcal{N}(0, \sigma^2(\mathbf{x}_i))$. For $\ell_{MSE}(\hat{p}, y) = (\hat{y} - y)^2$, the decomposition attains a familiar form:

Aleatoric-epistemic uncertainty decomposition for squared error regression

$$\mathbb{E}_{y|\mathbf{x}}[\mathbb{E}_{\mathcal{D}_N, \gamma}(\hat{y} - y)^2|\mathbf{x}] = \underbrace{\sigma^2(\mathbf{x})}_{\text{aleatoric}} + \underbrace{\text{Var}_{\mathcal{D}_N, \gamma}(\hat{y}|\mathbf{x}) + \text{bias}^2(\hat{y}|\mathbf{x})}_{\text{epistemic}}$$

We can further decompose the **variance** part using the law of total variance:

$$\text{Var}_{\mathcal{D}_N, \gamma}(\hat{y}|\mathbf{x}) = \underbrace{\mathbb{E}_{\mathcal{D}_N}[\text{Var}_{\gamma}(\hat{y}|\mathbf{x})|\mathcal{D}_N]}_{\text{procedural uncertainty}} + \underbrace{\text{Var}_{\mathcal{D}_N}(\mathbb{E}_{\gamma}[\hat{y}|\mathbf{x})|\mathcal{D}_N)}_{\text{data uncertainty}}.$$

Second-order representations and disentanglement

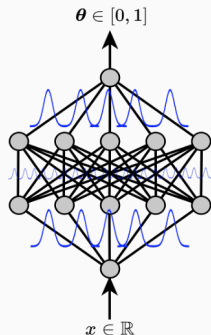
Uncertainty is modeled via a distribution

$$q(\boldsymbol{\theta}|\mathbf{x}) : \mathcal{X} \rightarrow \mathbb{P}(\Theta).$$

over the first-order parameters $\boldsymbol{\theta}$. The predictive distribution is obtained by Bayesian model averaging:

$$\hat{p}(y|\mathbf{x}) = \int_{\Theta} p(y|\boldsymbol{\theta}, \mathbf{x}) q(\boldsymbol{\theta}|\mathbf{x}) d\boldsymbol{\theta}.$$

Note: We usually approximate $q(\boldsymbol{\theta}|\mathbf{x})$ by samples $\hat{\boldsymbol{\theta}}_1, \dots, \hat{\boldsymbol{\theta}}_M$. In the regression case, $\hat{\boldsymbol{\theta}}_i = (\hat{\mu}_i, \hat{\sigma}_i^2)$.



Second-order representations and disentanglement

We model uncertainty via a distribution $q(\boldsymbol{\theta}|\mathbf{x})$ over the first-order parameters $\boldsymbol{\theta}$. The predictive distribution is obtained by **Bayesian model averaging**:

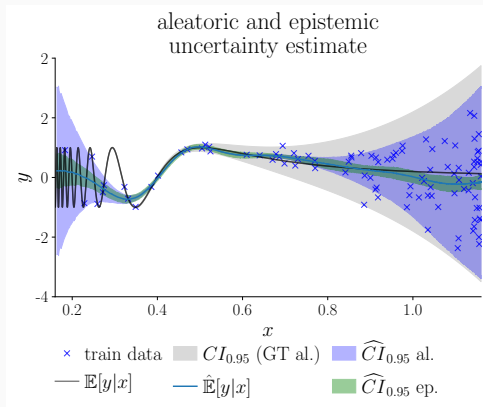
$$\hat{p}(y|\mathbf{x}) = \int_{\Theta} p(y|\boldsymbol{\theta}, \mathbf{x}) q(\boldsymbol{\theta}|\mathbf{x}) d\boldsymbol{\theta}$$

2. Disentanglement: Variance-based decomposition

The total predictive variance of $\hat{y}$ is commonly decomposed into two components:

$$\text{Var}(\hat{y}|\mathbf{x}) = \underbrace{\mathbb{E}_{\boldsymbol{\theta} \sim q} [\text{Var}(y|\mathbf{x}, \boldsymbol{\theta})]}_{\text{aleatoric estimate}} + \underbrace{\text{Var}_{\boldsymbol{\theta} \sim q} (\mathbb{E}[y|\mathbf{x}, \boldsymbol{\theta}])}_{\text{epistemic estimate}}$$

High bias leads to false estimation of epistemic uncertainty.



Synthetic example: predicted AU (blue) and EU (green) of a heteroscedastic GP model

Predicted variance of second-order distributions do not take bias into account.

Synthetic example:

$$x_i \sim \text{Beta}(1.2, 0.5),$$

$$y_i = \sin\left(\frac{1}{5(x_i + 0.16)^3}\right) + \epsilon_i$$

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2(x_i))$$

with $\sigma^2(x) = x^4$.

The reference distribution: a frequentist evaluation approach

Definition 1 (adapted from previous paper⁷)

The reference distribution $q_{N,\gamma}$ describes the probability of obtaining parameters θ given the randomness in **data sampling** (P^N) and **procedure** (P^γ):

$$q_{N,\gamma}(\theta|\mathbf{x}) := \int_{\Gamma} \int_{\mathcal{X} \times \mathcal{Y}} \mathbb{I} \left[\hat{\theta}_{N,\gamma}(\mathbf{x}) = \theta \right] dP^N dP^\gamma$$

Empirical approximation: We estimate $q_{N,\gamma}$ by nested resampling of:

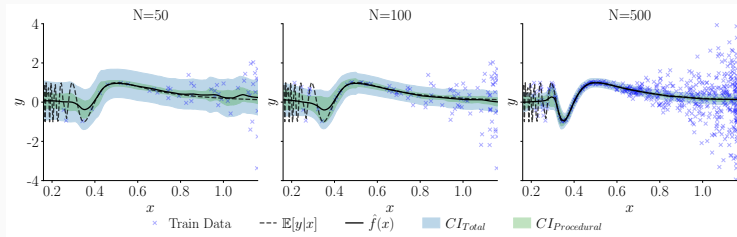
1. datasets $\mathcal{D}_N$ (data uncertainty)
2. different random seeds γ (procedural uncertainty).

Recent results⁸ show estimation of $q_{N,\gamma}(\theta|\mathbf{x})$ based on bootstrapping.

⁷Jürgens et al., “Is Epistemic Uncertainty Faithfully Represented by Evidential Deep Learning Methods?”

⁸Jain et al., “Frequentist Validity of Epistemic Uncertainty Estimators”.

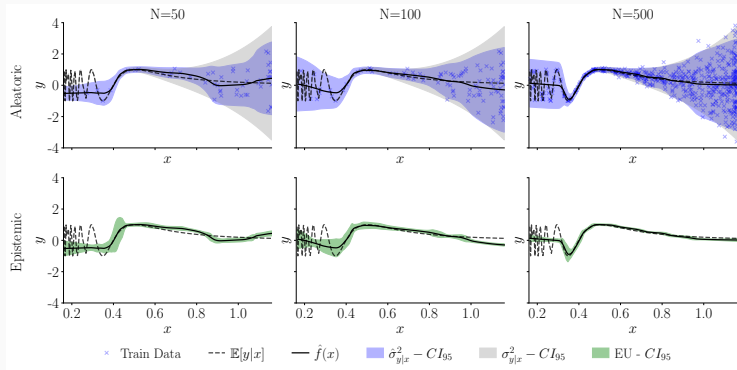
Decomposing the reference distribution



Observations:

- **Variance shrinks with N :** Both **data** and **procedural** uncertainty decrease with increasing sample size.
- **High bias in data-sparse region:** The high model bias leads to a misleadingly low estimate of epistemic uncertainty.

Experimental results - synthetic data



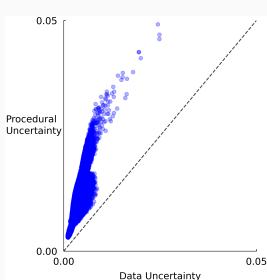
(a) Deep Ensemble estimates

Observations:

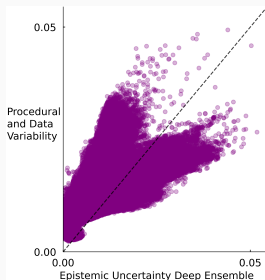
- overestimation of true aleatoric uncertainty in regions with high bias
- underestimation of epistemic uncertainty: most methods capture purely procedural part of variance

Experimental results - real data

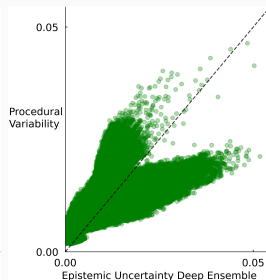
- New York taxi trip duration dataset published in Kaggle competition
- 1,206,737 observations → divide into 100 disjoint datasets to estimate reference distribution
- compare with estimates on models trained on subsets of data



(a) data vs procedural uncertainty, reference distribution



(b) Deep Ensemble epistemic estimate vs data and procedural uncertainty



(c) Deep Ensemble epistemic estimate vs procedural uncertainty

Conclusion and outlook

1. **Taxonomy matters:** A fine-grained taxonomy of epistemic uncertainty is useful for proper evaluation.
2. Second-order methods primarily capture procedural uncertainty, ignoring data uncertainty and model bias.
3. High model bias can lead to confusion of aleatoric/epistemic uncertainty.



arXiv Preprint

Outlook

- "bias-aware" uncertainty quantification: explicitly model systematic bias.
- usage of simulation based protocols to validate which source a method captures.

Thank you!